

20250701 日报

今日工作内容

1、学习 actor 模型知识。

- 定义：actor 是 actor 模型中的基本计算单元，它是一个独立的、并发的计算实体，具有自己的状态和行为，并通过接收和处理消息来改变其状态和行为；
- 核心思想：独立维护隔离状态，并基于消息传递实现异步通信；
- 主要组件：pid（唯一性）、state（每个 actor 的描述信息）、behavior、mailbox（就是消息队列）、children（actor 可以创建子 actor）、supervisor（父 actor 处理子 actor 的策略，比如重启策略、停止策略等）；
- 为什么要用 actor 模型：最主要原因就是 actor 通过消息传递和状态隔离解决了传统共享内存模型的缺陷（数据竞争时需求加锁（耗资源）、加锁可能出现死锁等情况）；
- actor 缺点：actor 系统中各个 actor 是异步通信的，因此消息是无法保证严格顺序的；
- actor 不需要锁的关键技术是：状态私有化（物理隔离）：每个 actor 维护私有状态，其他 actor 无法直接访问或修改，只能通过发送消息请求变更、消息队列（FIFO 串行处理）：每个 actor 的邮箱按 FIFO 顺序处理消息，每次仅处理一条消息，相当于天然互斥锁，即不会出现数据竞争（同时一起操作共享资源才会数据竞争）、异步通信（无阻塞）：actor 发送消息后无需等待响应，可以继续处理其他任务，避免了线程阻塞和死锁风险。

为了更好的说明 actor 模型的并发优势，我模拟了一个并发增加数值的场景并做了它与传统加锁方法的时间对比结果。

```

// Actor 模型方案
type Increment struct{ Amount int }

// CounterActor 管理计数器状态
type CounterActor struct{ value int }

func (c *CounterActor) Receive(ctx actor.Context) {
    switch msg := ctx.Message().(type) {
    case *Increment:
        c.value += msg.Amount // 无锁修改状态
    }
}

func main() {
    system := actor.NewActorSystem()
    counters := make([]*actor.PID, 1000)

    // 创建 1000 个计数器 Actor
    for i := 0; i < 1000; i++ {
        counters[i] =
system.Root.Spawn(actor.PropsFromProducer(func() actor.Actor {
        return &CounterActor{}
    })))

    start := time.Now()
    for i := 0; i < 10000; i++ { // 10000 个用户
        go func(id int) {
            for j := 0; j < 500; j++ { // 每个计数器执行 500 次增加操作
                // 向指定计数器发送增加消息
                system.Root.Send(counters[id%1000],
&Increment{Amount: 1})
            }
        }(i)
    }

    fmt.Printf("Actor 模型耗时: %v\n", time.Since(start))
}

```

=>Actor 模型耗时: 76.876529ms

// 加锁方案

从结果可以看出，在高并发且需低延迟的场景中 Actor 模型有着显著的优势。

今日所遇问题：

1. 合并 silk_song 时出现合并冲突，具体原因是：之前做首充续充需求时改了 yaml 文件，但是没有同步到本地中，因此出现了冲突。
2. 合并代码时蓝盾审核不通过：因为导出函数和导出结构体没有添加注释。

明日待办：

1. 研究基于 proto.actor 构建跨节点的分布式 Actor 系统，学习 grain=分布式虚拟 actor。